Intrinisically Typed Syntax That Works (Fresh Perspective)

ANONYMOUS AUTHOR(S)

Intrinsically typed syntax promises elegant mechanizations: well-typed terms are represented by well-typed data, substitution preserves typing by construction, and type safety proofs collapse to straightforward induction. For simple calculi such as the simply-typed lambda calculus this promise is fulfilled. For realistic systems, however, the technique often becomes impractical. In dependently typed proof assistants, intrinsically typed encodings of polymorphic calculi lead to pervasive type-changing equalities: renamings and substitutions indexed by other substitutions force proofs to transport terms across equal types that are not definitionally equal. This phenomenon—informally known as transfer hell—causes even routine metatheoretic arguments to be dominated by administrative reasoning.

This fresh perspective shows how to make intrinsically typed syntax work in practice for polymorphic calculi. We give extacts of a mechanization of the metatheory of System F and extend it to System Fω in Agda while avoiding explicit reasoning about transports almost entirely. The key is to internalize the equational theory of the σ-calculus for substitutions and make it definitional. We rely on proved σ-calculus laws for a functional representation of substitutions and install them into Agda's rewrite mechanism, obtaining canonical behavior for substitution and renaming without explicit transports. The main technical challenge is ensuring that the resulting rewrite system remains terminating and confluent together with Agda's built-in reductions.

The use of rewriting dramatically simplifies proofs of metatheoretic properties: typing preservation becomes definitional, substitution lemmas disappear, and even well-behaved type conversions can be handled intrinsically. Our experience suggests a general methodology: when intrinsically typed syntax generates transports, the correct solution is often not more lemmas but better definitional equality. We propose some practical guidelines towards this methodology.

ACM Reference Format:
Anonymous Author(s). 2018. Intrinisically Typed Syntax That Works (Fresh Perspective). *ACM Trans. Program. Lang. Syst.* 37, 4, Article 111 (August 2018), 17 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Intrinsically typed syntax is widely recommended for mechanizing type systems in dependently typed proof assistants. By representing well-typed terms as well-typed data, typing invariants hold by construction as in a Church-style presentation, i.e., making typing preservation trivial. For small calculi this approach works beautifully. However, for realistic polymorphic calculi such as System F, mechanizations often become complicated: conceptually simple proofs become cluttered with transports across equal types and administrative equalities.

The simply-typed lambda calculus is the prime example where intrinsically typed syntax works like a charm. Figure 1 shows typical definitions of the syntax of types, type contexts, variables, and expressions, which are indexed by contexts and types (left column). Building on these definitions it is straightforward to define a small-step operational semantics and proving type safety. This is because subject reduction holds by construction and progress follows by a simple inductive proof (right column). The style of these definitions is adapted from the textbook by Kokke et al. [5].

ACM 1558-4593/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

```
data Type : Set where
  𝟙    : Type
  _⇒_ : Type → Type → Type

data Ctx : Set where
  ∅    : Ctx
  _▷_ : Ctx → Type → Ctx

data _∋_ : Ctx → Type → Set where
  here  : (Γ ▷ T) ∋ T
  there : Γ ∋ T → (Γ ▷ U) ∋ T

data _⊢_ Γ : Type → Set where
  con : Γ ⊢ 𝟙
  var : Γ ∋ T → Γ ⊢ T
  lam : (Γ ▷ T) ⊢ U → Γ ⊢ (T ⇒ U)
  app : Γ ⊢ (T ⇒ U) → Γ ⊢ T → Γ ⊢ U

data _⟶_ {Γ} {T} : Γ ⊢ T → Γ ⊢ T → Set where
  β-lam : app {Γ = Γ} (lam e₁) e₂ ⟶ (e₁ [ e₂ ])
  ξ-app : e₁ ⟶ e₁′ → app e₁ e₂ ⟶ app e₁′ e₂

data Value {Γ} : Γ ⊢ T → Set where
  con : Value con
  lam : (e : (Γ ▷ T) ⊢ U) → Value (lam e)

data Progress : Γ ⊢ T → Set where
  done : Value e → Progress e
  step : e ⟶ e′ → Progress e

progress : (e : ∅ ⊢ T) → Progress e
progress con       = done con
progress (lam e)   = done (lam e)
progress (app e₁ e₂)
  with progress e₁
... | step x       = step (ξ-app x)
... | done (lam e) = step β-lam
```

Fig. 1. Call-by-name simply-typed lambda calculus

We elide the definition of the substitution operator _[_], as we discuss substitutions in detail in Section 2.2.

The simply-typed lambda calculus therefore represents the ideal scenario: intrinsic typing elides type preservation proofs, thus making type safety proofs so concise that they can serve as textbook examples [5]. Nothing comparable to transport reasoning arises in this development as only definitional properties of substitutions are required. The remainder of this paper explains why the same approach breaks down for polymorphic calculi and how to recover some of its simplicity.

Transfer hell is the colloquial term for being forced to apply type-changing lemmas to terms that are already known to be equal. The root problem is that substitutions that are propositionally equal are not definitionally equal, so terms must repeatedly be transported across equal types. This situation is annoying because proof assistants like Agda or Coq plainly refuse to let users even state this equality.[1] This phenomenon appears when substitutions themselves carry dependent typing information. In System F and Fω, expression substitutions depend on type substitutions and type substitutions depend on renamings; composing them yields types that are equal only propositionally, forcing transports throughout proofs. One workaround is to state definitions and proofs with explicit equivalence relations, but this practice is cumbersome, inefficient, and far away from the holy grail of definitional equality, which just works.

To our knowledge, previous intrinsically typed mechanization of System Fω (e.g., [2]) follow a natural design: expression syntax is indexed by typing derivations and substitutions are indexed by substitutions at the type level. While elegant at the level of definitions, this design makes proofs brittle. Because expression substitutions depend on type substitutions, and type substitutions depend on renamings, which again depend on type renamings, routine equalities require repeated transport across equal types, leading to directly to transfer hell.

[1] A situation that can be addressed using heterogeneous equality introduced in Conor McBride's thesis [8]. Benton et al. [1] discuss this approach in the context of an adequacy proof for a logical relation.

The message of this pearl is simple: when intrinsically typed syntax becomes complicated, the problem is often not the encoding of terms but the definition of equality on substitutions. Strengthening definitional equality can simplify metatheory more effectively than adding automation or additional lemmas.

In this paper, we examine the transfer hell arising by intrinsically-typed syntax encodings of System F and System Fω. Specifically, we consider the problems arising with type-renaming-indexed renamings and type-substitution-indexed substitutions. Our solution is to strengthen definitional equality rather than to add more lemmas. We make substitution compute canonically by internalizing the equational theory of substitutions (the σ-calculus) as definitional equality using Agda's rewriting mechanism. As a consequence, extensionally equal substitutions normalize to the same form and transports disappear from routine proofs. This approach is inspired by the tactics of Autosubst [11–13], but internalized by rewriting [4]. We build on a standard functional representation of renamings and substitutions [1], prove the lemmas underlying the equational laws of the σ-calculus, and then inject these equations into Agda's rewrite engine, i.e., make the σ-calculus equations definitional. The technical challenge is to ensure that the added equations remain terminating and confluent together with the proof assistant's built-in definitional equality.

We first exercise this program for plain System F in Section 2, significantly simplifying the transfer-ridden approach used by Saffrich et al. [10], and then apply the same techniques to System Fω in **??**. In the latter, the main difference to previous work [2] lies in our intrinsic treatment of type conversion, which leads to significant simplifications. In particular, in our developments, transport lemmas disappear, subject reduction holds by construction, and type conversion in System Fω is handled intrinsically.

In summary, intrinsically typed syntax fails for polymorphic calculi because substitutions lack canonical equality; making them compute canonically restores the original promise of intrinsic typing.

Contributions

This functional pearl demonstrates how to mechanize polymorphic calculi using intrinsically typed syntax without pervasive transport reasoning.

- We identify the fundamental cause of "transfer hell" in intrinsically typed encodings of System F and System Fω: renamings and substitutions indexed by other substitutions produce equal types that are not definitionally equal.
- We show how to eliminate these transports by installing the equational laws of the σ-calculus for substitutions as definitional equality using Agda's rewriting mechanism, taking care to make the equations compatible with the built-in definitional equality.
- We demonstrate that the resulting rewrite system is compatible with Agda's definitional equality (in particular, terminating and confluent) for the functional representation of substitutions and verify that the resulting rewrite system is terminating and confluent with respect to the proof assistant's definitional equality.
- We mechanize parts of the metatheory of System F and extend it to System Fω, obtaining substantially simpler proofs and an intrinsic treatment of type conversion for the latter.
- From this experience we extract practical guidelines for designing intrinsically typed syntax in dependently typed proof assistants.

```
data Type (n : Nat) : Set where
  ‘_    : Var n → Type n
  ∀α    : Type (1 + n) → Type n
  _⇒_ : Type n → Type n → Type n
```

```
_ : Type 0 - a closed type:  ∀α. α→α
_ = ∀α (‘ zero ⇒ ‘ zero)
_ : Type 0 - ∀αβ. α→β→α
_ = ∀α (∀α (‘ suc zero ⇒ ‘ zero ⇒ ‘ suc zero))
```

Fig. 2. Syntax of System F types (Agda definition left, examples right)

2 System F Types

2.1 Syntax

The syntax of System F types uses the standard de Bruijn encoding of variables by numbers. The type Type n (Figure 2) stands for the set of types with n type variables in scope. Henceforth, we call such n a *scope*. Variables for scope n are drawn from the type Var n, i.e., the set $\{0, \ldots, n-1\}$, written as zero, suc zero, suc (suc zero), ...and ranged over by α. The $\forall\alpha$-binder introduces a new type variable, and $\Rightarrow$ is the infix function type constructor. We let T range over types.

2.2 Renaming and Substitution

The most important operations on types are consistent renaming of type variables and substitution. Figure 3 contains the first step in defining renamings (left column) and substitutions (right column). Our definitions of renamings and substitutions in functional style are standard [1, 5, 9]. For now, we ignore the use of opaque. We defer the discussion of using it to Section 2.3.

A type renaming ζ maps variables from one scope n_1 to another n_2. We write the type of type renamings as Ren n_1 n_2. The definitions comprise (in order) weakening (wk^R), the identity renaming (id^R), extending a renaming with variable α ($\alpha \cdot^R \zeta$), applying a renaming to a variable ($\alpha \,\&^R\, \zeta$), composing two renamings ($\zeta_1 \fatsemi^R \zeta_2$), lifting a renaming (to move under a binder) ($\zeta \uparrow^R$), and applying a renaming to a type ($\mathsf{T}\,[\,\zeta\,]^R$).

For substitutions (right column of Figure 3) we proceed analogously. A type substitution η : Sub n_1 n_2 maps variables from scope n_1 to types defined over scope n_2. The definitions (in order) comprise lifting a renaming to a substitution ($\langle\, \zeta \,\rangle$), extending a substitution with a new type T ($\mathsf{T} \cdot^S \eta$), applying a substitution to variable ($\alpha \,\&^S\, \eta$), lifting a substitution (to move under a binder) ($\eta \uparrow^S$), applying a substitution to a type ($\mathsf{T}\,[\,\eta\,]^S$), and composing two substitutions ($\eta_1 \fatsemi^S \eta_2$).

2.3 Laws for Renaming and Substitution

This section presents the equalities that we *actually* use to compute with renamings and substitutions, rather than their defining equations in Figure 3. The opaque blocks enable us to control this behavior: Function symbols defined in an opaque block are usable outside the block, but they do not reduce! That is, the only reducing definitions in the renaming block are $\zeta \uparrow^R$ for lifting a renaming and application of a renaming to a type; in the substitution block, only the application of a substitution to a type reduces.

This selective reduction behavior is essential for our approach because we want to compute using the laws of the σ-calculus with first-class renamings on top of the thus defined symbols, rather than their definitions. Blocking their reduction is necessary because the equalities from σ-calculus interfere in a bad way with the standard definitional equalities. The original defining equalities are available for proofs: We can open the opaque definitions locally and proceed as usual.

Next we introduce equational laws of the σ-calculus with first-class renamings [11]. All of these equalities are supported by proofs using the definitions in Figure 3. The proofs are available in the supplement, but they are not included here for space reasons.

```
-- renamings
Ren : Nat → Nat → Set
Ren n₁ n₂ = Var n₁ → Var n₂

opaque
  -- weakening
  wkᴿ : Ren n (1 + n)
  wkᴿ = suc

  -- identity renaming
  idᴿ : Ren n n
  idᴿ α = α

  -- extend with new variable
  _·ᴿ_ : Var n₂ → Ren n₁ n₂ → Ren (1 + n₁) n₂
  (α ·ᴿ ζ) zero    = α
  (_ ·ᴿ ζ) (suc α) = ζ α

  -- apply renaming to variable
  _&ᴿ_ : Var n₁ → Ren n₁ n₂ → Var n₂
  α &ᴿ ζ = ζ α

  -- left-to-right composition
  _⨟ᴿ_ : Ren n₁ n₂ → Ren n₂ n₃ → Ren n₁ n₃
  (ζ₁ ⨟ᴿ ζ₂) α = ζ₂ (ζ₁ α)

-- lifting
_↑ᴿ : Ren n₁ n₂ → Ren (1 + n₁) (1 + n₂)
_↑ᴿ ζ = zero ·ᴿ (ζ ⨟ᴿ wkᴿ)

-- apply renaming to type
_[_]ᴿ : Type n₁ → Ren n₁ n₂ → Type n₂
(` α) [ ζ ]ᴿ      = ` (α &ᴿ ζ)
(∀α T) [ ζ ]ᴿ     = ∀α (T [ ζ ↑ᴿ ]ᴿ)
(T₁ ⇒ T₂) [ ζ ]ᴿ = (T₁ [ ζ ]ᴿ) ⇒ (T₂ [ ζ ]ᴿ)
```

```
-- substitutions
Sub : Nat → Nat → Set
Sub n₁ n₂ = Var n₁ → Type n₂

opaque
  -- lift renaming to substitution
  ⟨_⟩ : Ren n₁ n₂ → Sub n₁ n₂
  ⟨ ζ ⟩ α = ` (α &ᴿ ζ)

  -- extend with new type
  _·ˢ_ : Type n₂ → Sub n₁ n₂ → Sub (1 + n₁) n₂
  (T ·ˢ η) zero    = T
  (T ·ˢ η) (suc α) = η α

  -- apply substitution to variable
  _&ˢ_ : Var n₁ → Sub n₁ n₂ → Type n₂
  α &ˢ η = η α

  -- lifting
  _↑ˢ : Sub n₁ n₂ → Sub (1 + n₁) (1 + n₂)
  _↑ˢ η = (` zero) ·ˢ λ α → (η α) [ wkᴿ ]ᴿ

-- apply substitution to type
_[_]ˢ : Type n₁ → Sub n₁ n₂ → Type n₂
(` α) [ η ]ˢ      = α &ˢ η
(∀α T) [ η ]ˢ     = ∀α (T [ η ↑ˢ ]ˢ)
(T₁ ⇒ T₂) [ η ]ˢ = (T₁ [ η ]ˢ) ⇒ (T₂ [ η ]ˢ)

opaque
  -- left-to-right composition
  _⨟ˢ_ : Sub n₁ n₂ → Sub n₂ n₃ → Sub n₁ n₃
  (η₁ ⨟ˢ η₂) α = (η₁ α) [ η₂ ]ˢ
```

Fig. 3. Renamings and substitutions on types

The first group presents computation rules for substitutions.

```
beta-ext-zero : zero &ˢ (T ·ˢ η)   ≡ T
beta-ext-suc  : suc α &ˢ (T ·ˢ η)  ≡ α &ˢ η
beta-rename   : α &ˢ ⟨ ζ ⟩         ≡ ` (α &ᴿ ζ)
beta-comp     : α &ˢ (η₁ ⨟ˢ η₂)    ≡ (α &ˢ η₁) [ η₂ ]ˢ
beta-lift     : η ↑ˢ               ≡ (` zero) ·ˢ (η ⨟ˢ ⟨ wkᴿ ⟩)
```

Starting with the application of a substitution to a variable, there is one equation for each possible form of a substitution: extended substitution, lifted renaming, and composition of substitutions. The

final equation in this group characterizes lifting in terms of extension, weakening, and composition. It is *not* possible to use this equation to define lifting in Figure 3 as it would lead to circularity.

The second group contains laws about interactions between substitutions.

$$
\begin{array}{lll}
\text{associativity} & : (\eta_1 \fatsemi^S \eta_2) \fatsemi^S \eta_3 & \equiv \eta_1 \fatsemi^S (\eta_2 \fatsemi^S \eta_3) \\
\text{distributivity} & : (T \cdot^S \eta_1) \fatsemi^S \eta_2 & \equiv (T\,[\,\eta_2\,]^S) \cdot^S (\eta_1 \fatsemi^S \eta_2) \\
\text{interact} & : \langle\, \mathsf{wk}^R \,\rangle \fatsemi^S (T \cdot^S \eta) & \equiv \eta \\
\text{comp-id}_r & : \eta \fatsemi^S \langle\, \mathsf{id}^R \,\rangle & \equiv \eta \\
\text{comp-id}_l & : \langle\, \mathsf{id}^R \,\rangle \fatsemi^S \eta & \equiv \eta \\
\eta\text{-id} & : (\text{`}\,\mathsf{zero}\ \{n\}) \cdot^S \langle\, \mathsf{wk}^R \,\rangle & \equiv \langle\, \mathsf{id}^R \,\rangle \\
\eta\text{-law} & : (\mathsf{zero}\ \&^S\ \eta) \cdot^S (\langle\, \mathsf{wk}^R \,\rangle \fatsemi^S \eta) & \equiv \eta
\end{array}
$$

Composition is assoziative, it distributes over extension, weakening cancels extension, the identity substitution is a right and left identity, and we give two η-laws about the interaction of extension and weakening.[2]

There are analogous rules for renamings corresponding to the first two groups. These rules are omitted for space reasons.

The third group presents laws that govern the composition of different kinds of term traversal, that is, the behavior of the identity renaming and the four possible compositions of renamings and substitutions.

$$
\begin{array}{lll}
\text{identity}_r & : T\,[\,\mathsf{id}^R\,]^R & \equiv T \\
\text{compositionality}^{RS} & : (T\,[\,\zeta_1\,]^R)\,[\,\eta_2\,]^S & \equiv T\,[\,\langle\,\zeta_1\,\rangle \fatsemi^S \eta_2\,]^S \\
\text{compositionality}^{RR} & : (T\,[\,\zeta_1\,]^R)\,[\,\zeta_2\,]^R & \equiv T\,[\,\zeta_1 \fatsemi^R \zeta_2\,]^R \\
\text{compositionality}^{SR} & : (T\,[\,\eta_1\,]^S)\,[\,\zeta_2\,]^R & \equiv T\,[\,\eta_1 \fatsemi^S \langle\,\zeta_2\,\rangle\,]^S \\
\text{compositionality}^{SS} & : (T\,[\,\eta_1\,]^S)\,[\,\eta_2\,]^S & \equiv T\,[\,\eta_1 \fatsemi^S \eta_2\,]^S
\end{array}
$$

The final group contains contains laws that mediate between substitutions and renamings: A substitution which applies a lifted renaming is just applying the renaming; the composition of two lifted renamings is the lifting of their composition as renamings.

$$
\begin{array}{lll}
\text{coincidence} & : T\,[\,\langle\,\zeta\,\rangle\,]^S & \equiv T\,[\,\zeta\,]^R \\
\text{coincidence-comp} & : \langle\,\zeta_1\,\rangle \fatsemi^S \langle\,\zeta_2\,\rangle & \equiv \langle\,\zeta_1 \fatsemi^R \zeta_2\,\rangle
\end{array}
$$

In general ,the extraction of renamings from substitutions requires a dedicated solving strategy [13] and we have omitted some further coincidence laws for brevity.

Orienting these equations from left to right results in a confluent rewrite system for the σ-calculus that we install in Agda's computation engine using the REWRITE pragma [4]. Confluence of these rewrite equations is mechanically checked by Agda through using the flags `-rewriting -confluence-check`. From now on, all equations shown in this section Section 2.3 are definitional!

As a sanity check for the definitions, we prove in Figure 4 that lifting a substitution and the action of a substitution satisfy the functor laws. With σ-calculus rewriting installed, all these lemmas hold definitionally as evidenced by their proof using reflexivity (refl). The lemmas marked with $*$ correspond directly to laws from the σ-calculus.[3] The lifting laws are shown on the left, the application laws on the right. The analogous results for renamings follow exactly the same pattern.

[2]In the statement of η-id, we write zero $\{n\}$ to tell Agda's type checker that n is the scope of type variable zero. This notation supplies an *implicit argument* that Agda can usually infer.

[3]The naming of the lemmas is taken from Chapman et al. [2] to enable direct comparisons.

```
lifts*-id : (id^S {n} ↑^S) ≡ id^S
lifts*-id = refl

lifts*-comp : (η' ⨟^S η) ↑^S ≡ (η' ↑^S) ⨟^S (η ↑^S)
lifts*-comp = refl

sub*-id : T [ id^S ]^S ≡ T
sub*-id = refl

sub*-var : (` α) [ η ]^S ≡ α &^S η
sub*-var = refl  - *

sub*-comp : T [ η ⨟^S η' ]^S ≡ (T [ η ]^S) [ η' ]^S
sub*-comp = refl - *
```

Fig. 4. Functor laws for substitution proved

2.4 A Note on Safety

The theory underlying a proof assistant must be sound and consistent. Hence, it is a delicate act turning propositional equalities into definitional ones, for example by adding rewrite rules, because it can endanger both soundness and consistency. The combination of rewrite rules and rich dependent type theories has recently been explored by the addition of rewrite rules to Agda [3] and Rocq [6], and there is ongoing research on *local* rewrite rules [7].

Rewrite rules and the reductions of a dependently typed system interact in non-trivial ways. Hence, it is important to have criteria when it is safe to add rewrites. We justify making the equations from Section 2.3 definitional by appealing to the approach put forward for Rewriting Type Theory (RTT) [4], which is implemented in Agda. In RTT, the reduction relation is extended by turning propositional equalities into reduction rules: a term matched by the left-hand side of an equation reduces to the right-hand side, exactly like in Section 2.3. The two properties we wish to preserve when adding rewrite rules are subject reduction and consistency. In particular, the type checker assumes subject reduction and it would break without it.

To uphold subject reduction, the critical property of the newly added rewriting equations is that their *union with the existing reduction relation* is confluent. Agda already provides a plethora of confluent reduction rules: β, η (function and record expansion), δ (definition unfolding), and ι (dependent pattern matching clauses). Agda checks confluence of the extended system using a conservative set of simple syntactic criteria [**?**].

Some β and ι reductions arising from the functional definitions of renamings and substitutions in Figure 3 interfere with the intended rewrite equalities so that confluence breaks. For this reason, we explicitly hide these definitions inside an opaque block, so that only the equations we install with REWRITE define their reduction behaviour.

As a concrete example, consider the equation η-law: $(\mathsf{zero} \,\&^S\, \eta) \cdot^S (\langle\, \mathsf{wk}^R \,\rangle \,⨟^S\, \eta) \equiv \eta$. Without the opaque block, the underlying definition $(\eta_1 \,⨟^S\, \eta_2)\, \alpha = (\eta_1\, \alpha)\, [\, \eta_2 \,]^S$ would δ-unfold the inner composition $\langle\, \mathsf{wk}^R \,\rangle \,⨟^S\, \eta$ into the lambda $\lambda\, \alpha \to \eta\, (\mathsf{suc}\, \alpha)$, so that the entire left-hand side reduces to $(\eta\, \mathsf{zero}) \cdot^S (\lambda\, \alpha \to \eta\, (\mathsf{suc}\, \alpha))$. Two issues arise. First, the rewrite engine would no longer be able to match the left-hand side of η-law syntactically during reduction[4]. Second, even if the rewrite rule were admitted, confluence would fail, because the two reducts η (via the rewrite) and $(\eta\, \mathsf{zero}) \cdot^S (\lambda\, \alpha \to \eta\, (\mathsf{suc}\, \alpha))$ (via δ-unfolding) are propositionally but not definitionally equal. Agda has η-equality for function types, but $\cdot^S$ is defined by case analysis on its variable argument. The case-split therefore remains stuck on a neutral variable and cannot be collapsed back to η. Hiding the definition of the symbol $⨟^S$ (together with $\cdot^S$ and the other operators) inside an opaque block turns them into stuck symbols, restoring both matchability and confluence.

[4]To see why, note that η does not appear in pattern position here. A precise definition of pattern position is available in the Agda Wiki: https://agda.readthedocs.io/en/latest/language/rewriting.html#general-shape-of-rewrite-rules

```
weaken : Type n → Type (1 + n)
weaken T = T [ wk^R ]^R

_[_]* : Type (1 + n) → Type n → Type n
T [ T′ ]* = T [ T′ ·^S id^S ]^S

data Ctx : Nat → Set where
  ∅    : Ctx zero
  _▷_  : Ctx n → Type n → Ctx n
  _▷*  : Ctx n → Ctx (1 + n)

data _∋_ : Ctx n → Type n → Set where
  zero : (Γ ▷ T) ∋ T
  suc  : Γ ∋ T → (Γ ▷ T′) ∋ T
  suc* : Γ ∋ T → (Γ ▷*) ∋ weaken T
```

```
data Expr (Γ : Ctx n) : Type n → Set where
  `_   : Γ ∋ T →
         Expr Γ T
  λx   : Expr (Γ ▷ T1) T2 →
         Expr Γ (T1 ⇒ T2)
  _·_  : Expr Γ (T1 ⇒ T2) →
         Expr Γ T1 →
         Expr Γ T2
  Λα   : Expr (Γ ▷*) T →
         Expr Γ (∀α T)
  _·*_ : Expr Γ (∀α T) →
         (T′ : Type n) →
         Expr Γ (T [ T′ ]*)
```

Fig. 5. Type contexts, variables, and expressions

Consistency is preserved as long as the equalities we rewrite are instantiated with proofs. This is easy to see, because RTT can be translated to extensional type theory [**?**], a theory known to be consistent.

At present, Agda is the only mainstream proof assistant that supports the full feature set we require: functions whose reduction can be hidden inside opaque blocks, so they can be treated as symbols, instantiation of the rewrite equations over these symbols with proofs, and termination-independent confluence checking via the triangle property. Rocq currently lacks the ability to use defined functions as symbols inside rewrite equations and the ability to instantiate these equations with proofs, but aside from these differences our method transfers to Rocq directly using their rewrite rule implementation.

3 System F Expressions

3.1 Syntax

Working towards intrinsically-typed expressions, Figure 5 defines type weakening (weaken) and single substitution T [T′]* of a type T' for the innermost variable of T along with type contexts Γ, expression variables x, and the syntax of expressions e. Type contexts and expression variables follow the work of Chapman et al. [2]. A type context Γ : Ctx n keeps track of expression variables and type variables from scope n: the operator _▷_ adds an expression variable binding and the operator _▷* adds a type variable binding to Γ. Correspondingly, expression variables are implemented as typed de Bruijn indices with an extra case suc* to skip over type variables in the context. Skipping requires weakening of the type as it is used under one more type binding.

Expressions e : Expr Γ T are indexed by a context Γ and a type T with the intended reading as a typing judgment $\Gamma \vdash e : T$. As customary in an intrinsically typed setting, each expression constructor corresponds to a typing rule in System F. The rule for type application relies on single substitution for types.

3.2 Examples

As an example of System F expressions, we present some definitions for Church numerals. We start with their type and the encoding of zero. For readability, we define aliases for type variables like ‘α for ‘ Z, ‘β for ‘ (S Z), and analogously for expression variables and binders.

```
ℕᶜ : Type ∅ *                    zeroᶜ : Expr ∅ ℕᶜ
ℕᶜ = ∀α ((‘α ⇒ ‘α) ⇒ (‘α ⇒ ‘α))   zeroᶜ = Λα (λg (λx ‘x))
```

We continue with the constant one and the Church numeral for two.

```
oneᶜ : Expr ∅ ℕᶜ                 twoᶜ : Expr ∅ ℕᶜ
oneᶜ = Λα (λg (λx (‘g · ‘x)))    twoᶜ = succᶜ · (succᶜ · zeroᶜ)
```

The successor operation has a longer body and is all its glory.

```
succᶜ : Expr ∅ (ℕᶜ ⇒ ℕᶜ)
succᶜ = λx (Λα (λg (λx (‘g · (((((‘ (suc (suc (suc* zero)))) ·* ‘α) · ‘g) · ‘x)))))
```

3.3 Renaming and Substitution

The definition of a small-step operational semantics requires expression renamings and substitution to implement beta reduction for value abstractions and type abstractions. Renamings and substitutions are both indexed by type renamings and type substitutions, respectively. We skip the treatment of renamings as it is analogous to the one for substitutions and concentrate on explaining the latter.

Expression substitutions σ are indexed by type substitutions η. A substitution takes a variable x of type T in context Γ_1 and maps it to an expression typed in context Γ_2 adjusting its type according to the type substitution: $T[\eta]^S$. Notationally, we write $\eta \mid \ldots$ for any operation on substitutions that is indexed by type substitution η.

```
_|_⇒ˢ_ : Sub n₁ n₂ → Ctx n₁ → Ctx n₂ → Set
η | Γ₁ ⇒ˢ Γ₂ = ∀ T → (x : Γ₁ ∋ T) → Expr Γ₂ (T [ η ]ˢ)
```

Applying a substitution to a variable is just function application. Nevertheless, we use the same interface as for type substitutions to enable using the same rewriting machinery on the expression level again. For this definition, we include the typing of all symbols, which is similar for all further definitions and omitted because it can be inferred.

```
_|_&ˢ_ : ∀ {Γ₁ : Ctx n₁} {Γ₂ : Ctx n₂} (η : Sub n₁ n₂)
  → (x : Γ₁ ∋ T) → (σ : η | Γ₁ ⇒ˢ Γ₂) → Expr Γ₂ (T [ η ]ˢ)
η | x &ˢ σ = σ _ x
```

The identity substitution for expressions is straightforward to define. But this simplicity is due to the installed rewriting of type substitutions. Without the rewriting, the right hand side's type is T, but the expected type is $T[\mathsf{id}^s]^S$, which would require the transport lemma $\mathsf{comp\text{-}id}_r$.

```
Idˢ : idˢ | Γ ⇒ˢ Γ
Idˢ _ = λ x → ‘ x
```

Extending a substitution is standard: given an expression e and a substitution σ, we return a substitution that sends variable zero to e and otherwise resorts to σ.

```
_|_·ˢ_ : ∀ η → (e : Expr Γ₂ (T [ η ]ˢ)) → (σ : η | Γ₁ ⇒ˢ Γ₂) → η | (Γ₁ ▷ T) ⇒ˢ Γ₂
(_ | e ·ˢ σ) _ zero    = e
(_ | e ·ˢ σ) _ (suc x) = σ _ x
```

Due to the structure of contexts, we need two lifting lemmas when defining the application of a substitution to an expression, one for value bindings and another for type bindings. In both cases, the substitution σ needs to be adjusted (i.e., lifted) to accommodate the newly introduced binding. The former is standard: it maps the newly introduced variable to itself (zero) and weakens the images of σ accordingly. However, the definition of lifting requires a transport lemma to line up the type substitutions, which is applied by rewriting. The last part of the definition $\mathsf{id}^R \mid \ldots \mathsf{Wk}^R \ldots$ applies an expression renaming (weakening by $T[\eta]^S$) indexed by id^R to the output of σ.

```
_|_⇑ˢ_ : ∀ η → (σ : η | Γ₁ ⇒ˢ Γ₂) → ∀ T → η | (Γ₁ ▷ T) ⇒ˢ (Γ₂ ▷ (T [ η ]ˢ))
η | σ ⇑ˢ T = η | (‘ zero) ·ˢ λ _ x → id^R | (σ _ x) [ Wk^R (T [ η ]ˢ) ]^R
```

Lifting a substitution over a type binding is non-standard, but straightforward. The type substitution requires lifting, but no rewriting is needed.

```
_|_⇑ˢ* : ∀ η → (σ : η | Γ₁ ⇒ˢ Γ₂) → (η ↑ˢ) | (Γ₁ ▷*) ⇒ˢ (Γ₂ ▷*)
(η | σ ⇑ˢ*) = (η ⨟ˢ ⟨ wk^R ⟩) | (‘ zero) ·ˢ* λ _ x → wk^R | (σ _ x) [ wk^R* ]^R
```

These operations are sufficient to define the application of a substitution to an expression. The cases dispatch exactly to the operations that we introduced: applying a substitution to a variable, lifting over a value binding (case λx), lifting over a type binding (case $\Lambda\alpha$), the cases for application and type application are homomorphic.

```
_|_[_]ˢ : (η : Sub n₁ n₂) → (e : Expr Γ₁ T) → (σ : η | Γ₁ ⇒ˢ Γ₂) → Expr Γ₂ (T [ η ]ˢ)
η | (‘ x) [ σ ]ˢ     = η | x &ˢ σ
η | (λx e) [ σ ]ˢ    = λx (η | e [ η | σ ⇑ˢ _ ]ˢ)
η | (Λα e) [ σ ]ˢ    = Λα ((η ↑ˢ) | e [ η | σ ⇑ˢ* ]ˢ)
η | (e · e₁) [ σ ]ˢ  = (η | e [ σ ]ˢ) · (η | e₁ [ σ ]ˢ)
η | (e ·* T) [ σ ]ˢ = (η | e [ σ ]ˢ) ·* (T [ η ]ˢ)
```

3.4 Semantics

We are now in position to define a small-step operational semantics. As in the introduction, we opt for call-by-name for concision. The definition of single substitution, needed for beta reduction, is analogous to the one on types: we build the substitution by extending the identity substitution with the argument e'. As usual in intrinsically typed setting, this definition already proves that substitution preserves typing!

```
_[_] : Expr (Γ ▷ T′) T → Expr Γ T′ → Expr Γ T
e [ e′ ] = id^S | e [ id^S | e′ ·ˢ Id^S ]ˢ
```

In addition, we have to substitute a type into an expression to cater for beta reduction of type applications. This substitution is also implemented by an expression substitution indexed by a type substitution that performs the actual change.

```
_[*_*] : Expr (Γ ▷*) T → (T′ : Type n) → Expr Γ (T [ T′ ]*)
e [* T′ *] = (T′ ·ˢ id^S) | e [ id^S | T′ ·ˢ* Id^S ]ˢ
```

Due to the extended type substitution on the left, we also have to adapt the expression substitution on the right to the extended context structure. This adaptation is the type-level analogue to extension:

```
data Value : Expr Γ T → Set where
  λx  : (e : Expr (Γ ▷ T₁) T₂) → Value (λx e)
  Λα  : (v : Value e)          → Value (Λα e)

data Progress : Expr Γ T → Set where
  done : (v : Value e) → Progress e
  step : (e⟶e′ : e ⟶ e′) → Progress e

NoVar : Ctx n → Set
NoVar Γ = ∀ {T′} → ¬ (Γ ∋ T′)
```

```
progress : NoVar Γ → (e : Expr Γ T) → Progress e
progress nv (` x)   = ⊥-elim (nv x)
progress nv (λx e) = done (λx e)
progress nv (e · e₁)
  with progress nv e
... | done (λx e₂)    = step β-λ
... | step e⟶e′ = step (ξ-·₁ e⟶e′)
progress nv (Λα e)
  with progress (λ{ (suc* x) → nv x}) e
... | done v        = done (Λα v)
... | step e⟶e′ = step (ξ-Λ e⟶e′)
progress nv (e ·* T′)
  with progress nv e
... | done (Λα v) = step β-Λ
... | step e⟶e′ = step (ξ-·₁* e⟶e′)
```

Fig. 6. Progress for System F

```
_|_·ˢ*_ : ∀ η T → (σ : η | Γ₁ ⇒ˢ Γ₂) → (T ·ˢ η) | (Γ₁ ▷*) ⇒ˢ Γ₂
(_ | T ·ˢ* σ) _ (suc* x) = σ _ x
```

With substitution in place, the definition of the call-by-name small-step reduction relation is straightforward. As usual, such a definition of reduction for intrinsically-typed syntax implies subject reduction by construction.

```
data _⟶_ : Expr Γ T → Expr Γ T → Set where
  β-λ : (λx e₁ · e₂)  ⟶ (e₁ [ e₂ ])
  β-Λ : (Λα e ·* T′) ⟶ (e [* T′ *])
  ξ-·₁  : e₁ ⟶ e₁′ → (e₁ · e₂) ⟶ (e₁′ · e₂)
  ξ-·₁* : e ⟶ e′   → (e ·* T) ⟶ (e′ ·* T)
  ξ-Λ : e ⟶ e′   → (Λα e) ⟶ (Λα e′)
```

To obtain type safety, it remains to prove progress. Figure 6 presents the complete proof of progress for call-by-name System F using the structure from PLFA [5]. A value is either a lambda or a big lambda where the body is already a value. An expression fulfills progress if it is either a value (done) or if it can make a step (step). Due to the definition of value for big lambdas, we establish progress for contexts that may contain type bindings, but no value bindings. These contexts are characterized by function NoVar. The actual proof of progress is by induction on the structure of expressions. The overall structure of this proof closely follows a proof for System Fω [2]. Their proof is more complicated because their semantics is call-by-value and their language supports isorecursive types.

4 Laws for Renaming and Substitution

Proving type safety for System F *requires* exercising the substitution theory at the *type level*. Without installing its laws as definitional equalities, many definitions become more cumbersome because transport lemmas have to be inserted to equalize the types of (Agda) expressions.

While proving type safety does *not* require exercising the same theory at the *expression level*, it is still good practice to verify functorial and monadic properties of expression substitutions. Moreover, if we were to move on to constructing and proving properties of logical relations, then these properties would be needed [10].

As an example, we examine the composition of two expression substitutions. Its definition involves two type substitutions η_1 and η_2 that index the corresponding expression substitutions σ_1 and σ_2. The composed expression substitution is indexed by the composition of η_1 and η_2. Type checking this definition already exercises the laws for composition of type substitutions.

```
_,_|_⨟ˢ_ : ∀ η₁ η₂ → η₁ | Γ₁ ⇒ˢ Γ₂ → η₂ | Γ₂ ⇒ˢ Γ₃ → (η₁ ⨟ˢ η₂) | Γ₁ ⇒ˢ Γ₃
(_ , _ | σ₁ ⨟ˢ σ₂) _ x = _ | (σ₁ _ x) [ σ₂ ]ˢ
```

Here is the statement of functoriality of expression substitution: applying two substitutions one after the other to an expression is the same as applying their composition. This statement is a very prominent example of transfer hell: the types of the left and right side of the equality are different and would require a transfer without our rewriting machinery in place.

```
Compositionalityˢˢ : ∀ (e : Expr Γ₁ T) (η₁ : Sub n₁ n₂) (η₂ : Sub n₂ n₃) (σ₁ : η₁ | Γ₁ ⇒ˢ Γ₂) (σ₂ : η₂ | Γ₂ ⇒ˢ Γ₃)
  η₂ | (η₁ | e [ σ₁ ]ˢ) [ σ₂ ]ˢ ≡ (η₁ ⨟ˢ η₂) | e [ η₁ , η₂ | σ₁ ⨟ˢ σ₂ ]ˢ
```

We show the proof of this property just do demonstrate that type adjustments for type substitutions can be ignored and we can concentrate on the gist of the proof, rather than being overwhelmed by transfers. The remaining fly in the ointment is the explicit passing of the type substitutions η_1 and η_2. We chose to make them explicit; while Agda can infer them in many places, there are equally many places where inference is not possible.

```
Compositionalityˢˢ (` x)      η₁ η₂ σ₁ σ₂ = refl
Compositionalityˢˢ (λx {T₁ = T₁} e) η₁ η₂ σ₁ σ₂ = cong λx (begin
    η₂ | η₁ | e [ η₁ | σ₁ ⇑ˢ T₁ ]ˢ [ η₂ | σ₂ ⇑ˢ (T₁ [ η₁ ]ˢ) ]ˢ
  ≡⟨ Compositionalityˢˢ e η₁ η₂ (η₁ | σ₁ ⇑ˢ T₁) (η₂ | σ₂ ⇑ˢ (T₁ [ η₁ ]ˢ)) ⟩
    (η₁ ⨟ˢ η₂) | e [ η₁ , η₂ | η₁ | σ₁ ⇑ˢ T₁ ⨟ˢ (η₂ | σ₂ ⇑ˢ (T₁ [ η₁ ]ˢ)) ]ˢ
  ≡⟨ cong ((η₁ ⨟ˢ η₂) | e [_]ˢ) (Lift-Dist-Compˢˢ σ₁ σ₂) ⟩
    (η₁ ⨟ˢ η₂) | e [ (η₁ ⨟ˢ η₂) | η₁ , η₂ | σ₁ ⨟ˢ σ₂ ⇑ˢ T₁ ]ˢ
  □)
Compositionalityˢˢ (Λα e)   η₁ η₂ σ₁ σ₂ = cong Λα (begin
    (η₂ ↑ˢ) | (η₁ ↑ˢ) | e [ η₁ | σ₁ ⇑ˢ* ]ˢ [ η₂ | σ₂ ⇑ˢ* ]ˢ
  ≡⟨ Compositionalityˢˢ e (η₁ ↑ˢ) (η₂ ↑ˢ) (η₁ | σ₁ ⇑ˢ*) (η₂ | σ₂ ⇑ˢ*) ⟩
    ((η₁ ⨟ˢ η₂) ↑ˢ) | e [ (η₁ ↑ˢ) , η₂ ↑ˢ | η₁ | σ₁ ⇑ˢ* ⨟ˢ (η₂ | σ₂ ⇑ˢ*) ]ˢ
  ≡⟨ cong (((η₁ ⨟ˢ η₂) ↑ˢ) | e [_]ˢ) (lift*-dist-Compˢˢ η₁ η₂ σ₁ σ₂) ⟩
    ((η₁ ⨟ˢ η₂) ↑ˢ) | e [ (η₁ ⨟ˢ η₂) | η₁ , η₂ | σ₁ ⨟ˢ σ₂ ⇑ˢ* ]ˢ
  □)
Compositionalityˢˢ (e₁ · e₂) η₁ η₂ σ₁ σ₂ = cong₂ _·_
  (Compositionalityˢˢ e₁ η₁ η₂ σ₁ σ₂)
  (Compositionalityˢˢ e₂ η₁ η₂ σ₁ σ₂)
Compositionalityˢˢ (e ·* T′) η₁ η₂ σ₁ σ₂ = cong (_·* (T′ [ η₁ ⨟ˢ η₂ ]ˢ))
  (Compositionalityˢˢ e η₁ η₂ σ₁ σ₂)
```

This proof relies on further properties, in particular Lift-Dist-Comp^{SS} (lifting a composition of substitutions is equal to composing their liftings) and lift*-dist-Comp^{SS} (its counterpart for lifting over type bindings), where the statement and proof would require explicit transfers. Essentially,

these properties form a "food chain" that stops at the composition properties of renamings with renamings and liftings, all of which follow similarly. The supplemental material gives the full story.

4.1 Comparison to a Proof in Transfer Hell

We now contrast our development with the same proof carried out without installing the σ-calculus equations as definitional equalities. As a point of comparison, we take the supplement of the logical relation construction of Saffrich et al. [10], which also mechanizes the compositionality properties for System F. To enable a direct comparison, we have aligned the surface syntax of their definitions with ours.

Inspecting the type of the compositionality lemma already exposes the core issue: because the σ-calculus equations are only propositional, the statement of the theorem must itself rely on subst to bridge equal-but-not-definitionally-equal indices.

```
Compositionality^SS-≡ :
  ∀ {η₁ : n₁ →^S n₂} {η₂ : n₂ →^S n₃}
    {Γ₁ : Ctx n₁} {Γ₂ : Ctx n₂} {Γ₃ : Ctx n₃}
    (e : Expr n₁ Γ₁ T)
    (σ₁ : η₁ | Γ₁ ⇒^S Γ₂) (σ₂ : η₂ | Γ₂ ⇒^S Γ₃) →
  let sub = subst (Expr n₃ Γ₃) (compositionality^SS T η₁ η₂) in
  sub (η₂ | (η₁ | e [ σ₁ ]^S) [ σ₂ ]^S) ≡ (η₁ ⨟^S η₂) | e [ η₁ , η₂ | σ₁ ⨟^S σ₂ ]^S
```

Every recursive call to the induction hypothesis introduces a fresh application of subst, which then has to be distributed to the outside of the term and combined with the substs arising from neighboring subterms before the goal can be closed. This bookkeeping is further complicated by the fact that the other appearances of type substitution lemmas required in the proof interact with the transported equalities in non-trivial ways. We illustrate the resulting complexity by zooming in on a single induction case—type application—of their compositionality lemma.

A first observation is that our development already saves one subst in the very definition of substitution application. The type application case requires the swap lemma of type $(T\,[\,T'\,]^*)\,[\,\eta\,] \equiv (T\,[\,\eta \uparrow^S\,])\,[\,T'\,[\,\eta\,]\,]^*$ to coerce the term to the expected type, leading to the following definition.

```
η | (_·*_ {T = T} e T') [ σ ]^S = subst (Expr _ _)
                                    (sym (swap^SS η T T'))
                                    ((η | e [ σ ]^S) ·* (T' [ η ]^S))
```

With this in place, we can examine the type application case of the compositionality proof itself. The authors employ heterogeneous equality [8] to avoid some of the administrative reasoning otherwise forced by transfer hell. Without heterogeneous equality, the proof would be considerably worse still, requiring several auxiliary lemmas that explicitly shuffle applications of subst around. Thus, the version below already constitutes one of the simplest forms of the argument.

```
let
  F₂ = Expr n₂ Γ₂ ; E₂ = sym (swap^SS η₁ T T')          ; sub₂ = subst F₂ E₂
  F₃ = Expr n₃ Γ₃ ; E₃ = sym (swap^SS (η₁ ⨟^S η₂) T T') ; sub₃ = subst F₃ E₃
  F₅ = Expr n₃ Γ₃ ; E₅ = sym (swap^SS η₂ (T [ η₁ ↑^S ]) (T' [ η₁ ]^S)) ; sub₅ = subst F₅ E₅
in
R.begin
  η₂ | (η₁ | (e ·* T') [ σ₁ ]^S) [ σ₂ ]^S
R.≅⟨ refl ⟩                                              - (1)
```

```
  η₂ | (sub₂ ((η₁ | e [ σ₁ ]ˢ) ·* (T′ [ η₁ ]ˢ))) [ σ₂ ]ˢ
R.≅⟨ H.cong₂ {B = Expr n₂ Γ₂} (λ _ ■ → η₂ | ■ [ σ₂ ]ˢ)                    - (2)
       (H.≡-to-≅ (sym E₂))
       (H.≡-subst-removable F₂ E₂ _) ⟩
  η₂ | ((η₁ | e [ σ₁ ]ˢ) ·* (T′ [ η₁ ]ˢ)) [ σ₂ ]ˢ
R.≅⟨ refl ⟩                                                              - (3)
  sub₅ ((η₂ | (η₁ | e [ σ₁ ]ˢ) [ σ₂ ]ˢ) ·* (T′ [ η₁ ]ˢ [ η₂ ]ˢ))
R.≅⟨ H.≡-subst-removable F₅ E₅ _ ⟩                                        - (4)
  (η₂ | (η₁ | e [ σ₁ ]ˢ) [ σ₂ ]ˢ) ·* (T′ [ η₁ ]ˢ [ η₂ ]ˢ)
R.≅⟨ Hcong₃ {B = λ ■ → Expr n₃ Γ₃ (∀α ■)} {C = λ _ _ → Type n₃ _ }       - (5)
       (λ _ ■₁ ■₂ → ■₁ ·* ■₂)
       (H.≡-to-≅ (lift-compositionality^SS T η₁ η₂))
       (Compositionality^SS e σ₁ σ₂)
       (H.≡-to-≅ (compositionality^SS T′ η₁ η₂)) ⟩
  ((η₁ ⨟ˢ η₂) | e [ η₁ , η₂ | σ₁ ⨟ˢ σ₂ ]ˢ) ·* (T′ [ η₁ ⨟ˢ η₂ ]ˢ)
R.≅⟨ H.sym (H.≡-subst-removable F₃ E₃ _) ⟩                               - (6)
  sub₃ (((η₁ ⨟ˢ η₂) | e [ η₁ , η₂ | σ₁ ⨟ˢ σ₂ ]ˢ) ·* (T′ [ η₁ ⨟ˢ η₂ ]ˢ))
R.≅⟨ refl ⟩                                                              - (7)
  (η₁ ⨟ˢ η₂) | (e ·* T′) [ η₁ , η₂ | σ₁ ⨟ˢ σ₂ ]ˢ
R.□
```

The proof proceeds in seven steps.

(1) Unfold the definition of expression substitution at the inner application of σ_1, exposing the subst inside.
(2) Remove the subst that sits inside the term under consideration, justified by heterogeneous equality.
(3) Reveal yet another subst introduced by the substitution application of σ_2
(4) Remove the subst on the outside.
(5) Apply the induction hypothesis while simultaneously coercing the index according to the compositionality law for type substitutions.
(6) Reinsert the subst demanded by the proof goal on the outside.
(7) Fold the definition of expression substitution to conclude the proof.

Of these, steps (2), (4), the type coercion in (5), and (6), as well as the translation into heterogeneous equality, are entirely absent from our rewriting-based proof, only the application of the induction hypothesis and the implicit (un)folding of definitions remain.

We complement this qualitative comparison with a quantitative one. The following table records the lines of code and number of characters required to mechanize the compositionality law for expression substitutions, together with the type-checking time of the respective developments.

	With Rewriting		**Transfer Hell**	
	LOC	Chars	LOC	Chars
Type substitution + proofs	217	12 246	285	12 584
Expression substitution + proofs	199	11 465	1 405	69 846
Total	**416**	**23 711**	**1 690**	**82 430**
Type-check time	4.49 s		9.51 s	

As expected, the amount of code required to formalize type substitution is roughly the same in both developments. For expression substitutions, however, the picture changes: the transfer-hell

version requires more than five times as many lines of code as the rewriting-based one. As a welcome side effect, the type-checking time is also reduced by about 50% when the σ-calculus equations are installed as rewrite rules.

4.2 Equational Theory for Expression Substitution

Having proven the functor laws for expression substitution, the hardest result has already been established. A natural question is whether we can go further and install a full equational theory for expression substitution as definitional equalities, just as we did at the type level. And in fact, we can. The σ-calculus for type-substitution-indexed expression substitutions is an extension of the one for type substitutions in **??**, and the same holds analogously for expression renamings.

Expression substitutions carry additional operators that have no counterpart at the type level. As a result, every substitution law from the type level that mentions extension or weakening splits into *two* expression-level laws: one for the *expression* variants ($\_\cdot^S\_$, wk^S), which is a direct transcription of the corresponding rule from Section 2.3, and one for the *type* variants ($\_\cdot^{S*}\_$, wk^{S*}), which is genuinely new. We illustrate this pattern on the η-id law. The expression variant

$$\eta\text{-}\mathsf{Id}^S : \langle\ \mathsf{id}^R\ \rangle \mid (\text{`}\ \mathsf{zero}) \cdot^S (\mathsf{Wk}^S\ T) \equiv \mathsf{Id}^S\ \{\Gamma = \Gamma \triangleright T\}$$

is a direct transcription of η-id from Section 2.3, whereas the type variant

$$\eta\text{-}\mathsf{Id}^{S*} : \langle\ \mathsf{wk}^R\ \rangle \mid (\text{`}\ \mathsf{zero}) \cdot^{S*} \mathsf{wk}^{S*} \equiv \mathsf{Id}^S\ \{\Gamma = \Gamma \triangleright *\}$$

asserts that extending the type-binding weakening with the topmost type variable ‘ zero yields the identity expression substitution. The same doubling applies to the remaining laws in which extension or weakening appears (η, interact, distributivity and some β laws).

The full equational system is included in the supplement, and Agda verifies its confluence. For the confluence check to go through, we had to artificially block reduction of the traversal functions $\_|\_[\_]^R$ and $\_|\_[\_]^S$ and re-install each of their defining clauses as a rewrite rule. This is necessary because, in the case of type application, the rewriting must perform two reduction steps simultaneously, as required by the triangle criterion: one step normalizes the type index, and one step reduces the expression itself, pushing the substitution inside.

4.3 Practical Guidelines for Other Intrinsic Developments

We have now made the equations of the σ-calculus definitional at two levels of the syntactic hierarchy: at the type level and at the expression level, where the latter *depends* on the former. In general, this strategy applies to arbitrarily tall *towers* of intrinsically typed syntaxes, in which each level's type indices themselves support substitution.

More precisely, consider a tower $T_0, T_1, \ldots, T_n$ of syntactic sorts in which the variables and substitutions of T_{i+1} are indexed by the contexts and substitutions of $T_0, \ldots, T_i$. The recipe proceeds bottom-up:

(1) Define T_0 together with its renamings and substitutions, and install the σ-calculus equations for T_0 as rewrite rules.
(2) For each i from 1 to n, define T_i *on top of* the already-definitional substitution machinery of $T_0, \ldots, T_{i-1}$, prove the corresponding σ-calculus laws for T_i, and install them as rewrite rules.

The key is that the laws for all lower levels are already definitional, when we define and prove the laws for level i. Otherwise, the types of the level-i operations would no longer match up on the nose, and we would be back in transfer hell.

The size of the equational theory grows with each level. A substitution at level i can be lifted under a binder of any sort $T_0, \ldots, T_{i-1}$, and each kind of binder induces its own family of laws, for

weakening and extension, and the corresponding interactions with composition and traversal. At the type level this contributes no additional laws. At the expression level we already saw a few new laws, namely the laws governing weakening and extension by types. At level three one would add even more laws, and so on. The rule count therefore grows linearly with the level, but each new family follows the same template as the previous one, so the proof effort per family stays constant.

As a concrete instance, the System $F^{\uparrow}_{C}$ [14] language features three levels: kinds, types, and expressions. Each level carries its own variables and substitution. Using our approach, it would be possible to construct an intrinsically typed representation for it and establish the substitution theory for all three levels uniformly, without any transfer reasoning.

5 Conclusion

Intrinsically typed syntax promises elegant mechanizations by making well-typed programs representable only as well-typed data, but for polymorphic calculi this promise is often undermined by pervasive transport reasoning. Our experience with System F and System Fω shows that the difficulty does not stem from binding structure or indexing itself, but from a mismatch between substitution and definitional equality: substitutions that are extensionally equal fail to compute to a canonical form. By internalizing the equational laws of the σ-calculus as definitional equality, substitutions become canonical and routine transports disappear from proofs. The resulting developments recover the simplicity seen in the simply-typed case while scaling to higher-order polymorphism and intrinsic type conversion. More broadly, this pearl suggests a practical guideline for mechanizing type systems in dependently typed proof assistants: when intrinsically typed encodings become proof-heavy, a promising strategy is often to strengthen definitional equality rather than introduce additional lemmas or automation.

References

[1] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *J. Autom. Reason.* 49, 2 (2012), 141–159. doi:10.1007/S10817-011-9219-0

[2] James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. 2019. System F in Agda, for Fun and Profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 255–297. doi:10.1007/978-3-030-33636-3_10

[3] Jesper Cockx. 2020. Type Theory Unchained: Extending Agda with Userined Rewrite Rules. In *25th International Conference on Types for Proofs and Programs (TYPES 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 175)*, Marc Bezem and Assia Mahboubi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1–2:27. doi:10.4230/LIPIcs.TYPES.2019.2

[4] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The Taming of the Rew: A Type Theory with Computational Assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341

[5] Wen Kokke, Jeremy G. Siek, and Philip Wadler. 2020. Programming Language Foundations in Agda. *Sci. Comput. Program.* 194 (2020), 102440. doi:10.1016/J.SCICO.2020.102440

[6] Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. 2024. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:18. doi:10.4230/LIPIcs.ITP.2024.26

[7] Yann Leray and Théo Winterhalter. 2026. Encode the Cake and Eat It Too: Controlling Computation in Type Theory, Locally. *Proc. ACM Program. Lang.* 10, POPL, Article 62 (Jan. 2026), 30 pages. doi:10.1145/3776704

[8] Conor McBride. 2000. Elimination with a Motive. In *Types for Proofs and Programs, International Workshop, TYPES 2000, Durham, UK, December 8-12, 2000, Selected Papers (Lecture Notes in Computer Science, Vol. 2277)*, Paul Callaghan, Zhaohui Luo, James McKinna, and Robert Pollack (Eds.). Springer, 197–216. doi:10.1007/3-540-45842-5_13

[9] Conor McBride. 2005. Type-preserving renaming and substitution. (2005). http://strictlypositive.org/ren-sub.pdf Unpublished manuscript.

[10] Hannes Saffrich, Peter Thiemann, and Marius Weidner. 2024. Intrinsically Typed Syntax, a Logical Relation, and the Scourge of the Transfer Lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development* (Milan, Italy) *(TyDe 2024)*. Association for Computing Machinery, New York, NY, USA, 2–15. doi:10.1145/

3678000.3678201

[11] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn Terms and Parallel Substitutions. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 359–374. https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf

[12] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq.* Ph. D. Dissertation. Saarland University, Saarbrücken, Germany. https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.).

[13] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions *(CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3293880.3294101

[14] Brent A. Yorgey, Stephanie Weirich, Julien Cretin, Simon Peyton Jones, Dimitrios Vytiniotis, and José Pedro Magalhães. 2012. Giving Haskell a promotion. In *Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation* (Philadelphia, Pennsylvania, USA) *(TLDI '12)*. Association for Computing Machinery, New York, NY, USA, 53–66. doi:10.1145/2103786.2103795